Structures de données en C++

Standard Template Library

La ***Standard Template Library*** (**STL**) est une bibliothèque C++, normalisée par l'ISO (document ISO/CEI 14882) et mise en œuvre à l'aide des templates.

En programmation informatique, les ***templates*** (en français **modèles**, parfois aussi appelés **patrons**) sont une particularité de la programmation en langage C++, qui autorise l'écriture d'un code sans considération envers le type des données avec lesquelles il sera finalement utilisé. Les templates introduisent le concept de programmation générique dans le langage.

Cette bibliothèque fournit :

- Un ensemble de classes conteneurs, telles que les vecteurs (`vector`), les tableaux associatifs (`map`), les listes chaînées (`list`), qui peuvent être utilisées pour contenir n'importe quel type de données à condition qu'il supporte certaines opérations comme la copie et l'assignation.
- Une abstraction des pointeurs : les itérateurs. Ceux-ci fournissent un moyen simple et élégant de parcourir des séquences d'objets et permettent la description d'algorithmes indépendamment de toute structure de données.
- Des algorithmes génériques tels que des algorithmes d'insertion/suppression, recherche et tri.
- Une classe `string` permettant de gérer efficacement et de manière sûre les chaînes de caractères.

La bibliothèque STL est principalement un ensemble d'objets et de méthodes standards pour C++. Pour résumer, la STL est un ensemble de structures pouvant contenir n'importe quel type de données. Cela veut donc dire que, dans les piles et les files, on peut stocker tout type de données (entiers, réels, chaînes de caractères, ...) mais un seul type par structure : si je déclare une pile d'entiers, je ne pourrais pas empiler des chaînes de caractères !

Conteneurs

`<list>` : Fournit une classe patron `std::list` représentant une liste doublement chaînée.
`<queue>` : Fournit une classe patron `std::queue`, une file, et `std::priority_queue.`
`<stack>` : Fournit la classe patron `std::stack`, représentant une pile.
`<vector>` : Fournit la classe patron `std::vector,` un tableau dynamique.

Structures de données élémentaires

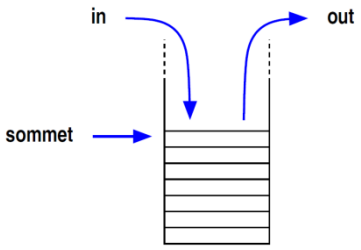
Les piles et les files sont des ensembles dynamiques pour lesquels l'élément à traiter est défini par la nature intrinsèque de l'ensemble.

Dans une **pile**, l'élément supprimé est le dernier inséré : la pile met en œuvre le principe **dernier entré, premier sorti**, ou **LIFO** (Last-In, First-Out). De même, dans une **file**, l'élément supprimé est

toujours le plus ancien ; la file met en œuvre le principe **premier entré, premier sorti**, ou **FIFO** (First-In, First-Out). Il existe plusieurs manières efficaces d'implémenter des piles et des files dans un ordinateur. Dans ce cours, nous allons montrer comment les implémenter en C++ à l'aide de la **STL** pour (***S**tandard **T**emplate **L**ibrary*).

Les Piles avec La STL C++

L'opération INSÉRER dans une pile est souvent appelée EMPILER et l'opération SUPPRIMER, qui ne prend pas d'élément pour argument, est souvent appelée DÉPILER.



Ces noms font allusion aux piles rencontrées dans la vie de tous les jours, comme les piles d'assiettes. L'ordre dans lequel les assiettes sont dépilées est l'inverse de celui dans lequel elles ont été empilées, puisque seule l'assiette supérieure est accessible.

```
1  /*
2   * File:   main.cpp
3   * Author: ACER
4   *
5   * Created on 23 février 2017, 20:07
6   */
7  //*****************************************************************************
8  #include <iostream>
9  #include <time.h>
10 #include <stack>    // pile
11 #include <cstdlib>  // pour rand srand
12 #define Nmax 12
13 using namespace std;
14 //*****************************************************************************
15 int nombreAleatoire(int nombreMax) // fonction qui génère un nombre aléatoire
16                                   // inférieur à nombreMax
17       {
18          return (rand() % nombreMax);
19       }
20  //*****************************************************************************
21 int main()
22 {
23     stack<int> Pile;
24     int element;
25     cout << "\n......... EMPILER les elements dans la Pile .........\n\n";
26     srand(time(NULL));  // initialisation de rand
27
28     cout << "\t";
29     for (int i=1; i<=Nmax; ++i)
30     {
31         element=nombreAleatoire(99);
32         Pile.push(element);
33         cout << '('<< element  << ')'<< ' ';
34     }
35     cout << "\n\n\n\ttaille de la pile = " << Pile.size() << endl;
36     cout << "\n\n........ DEPILER les elements de la Pile (LIFO) ........\n\n";
37     cout << "\t";
38     int sommet;
39     while (!Pile.empty())
```

```
40      {
41          sommet=Pile.top();
42          cout << '(' << sommet <<')' << ' ';
43          Pile.pop();
44      }


45        cout << endl <<"\n............................... Fin du Programme! ..."
46                << endl;
47        cin.get();
48        return 0;
49  }
50  //*****************************************************************************
```

Résultat de l'exécution : les valeurs empilées son générer aléatoirement entre `0` et `99`.

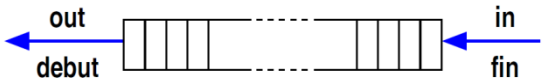
```
..... EMPILER les éléments dans la Pile ......

 37 41 15 97 27 75 51 28 57 27

 taille de la pile = 10

..... DEPILER les  éléments de la Pile (LIFO) .....

 27 57 28 51 75 27 97 15 41 37

... Fin du Programme! ...
```

Voici les opérations possibles sur une pile :

- déterminer la taille de la pile : `Pile.size()`;
- déterminer si la pile est vide ou non : `Pile.empty()` ;
- ajouter une nouvelle donnée : `Pile.push(element)` ;
- consulter la dernière donnée empilée : `Pile.top()` ;
- supprimer la dernière donnée ajoutée : `Pile.pop()` .

Les Files avec la STL C++

On appelle ENFILER l'opération INSÉRER sur une file et on appelle DÉFILER l'opération SUPPRIMER ; à l'instar de l'opération de pile DÉPILER, DÉFILER ne prend pas d'argument. La propriété FIFO d'une file la fait agir comme une file à un guichet. La file comporte une ***tête*** et une ***queue***. Lorsqu'un élément est enfilé, il prend place à la queue de la file. L'élément défilé est toujours le premier en tête.




```
1    #include <iostream>
2    #include <queue>
3    #include <time.h>
4    #include <cstdlib> // pour rand srand
5
6    using namespace std;
7
8   //*****************************************************************************
9     int nombreAleatoire(int nombreMax) // fonction qui génère un nombre aléatoire
10                                       // inférieur à nombreMax
11       {
12          return (rand() % nombreMax);
13       }
14  //*****************************************************************************
15
16   int main()
17   {
18       queue<int> File;
19       int element;
20       cout << "\n..... ENFILER les elements dans la File .....\n\n";
21       srand (time(NULL));  // initialisation de rand
22       for (int i=1; i<=10; ++i)
23                                {
24                                    element=nombreAleatoire(99);
25                                    File.push(element);
26                                    cout << ' ' << element;
27                                }
28
29       cout << "\n\n taille de la File = " << File.size();
30       cout << "\n\n ..... DEFILER les elements de la File (FIFO) .....\n\n";
31       int tete;
32       while (!File.empty())
33                                {
34                                    tete=File.front();
35                                    cout << ' ' << tete;
36                                    File.pop();
37                                }
38       cout << endl <<"\n... Fin du Programme! ..." << endl;
39       cin.get();
40       return 0;
41   }
42   //*****************************************************************************
```

Résultat de l'exécution : les valeurs enfilées son générer aléatoirement entre `0` et `99`.

```
..... ENFILER les éléments dans la File .....
```

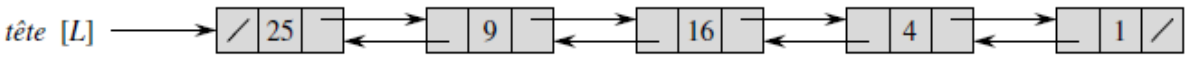
```
 15 13 1 6 65 75 7 81 42 93

 taille de la File = 10

 ..... DEFILER les éléments de la File (FIFO) .....

 15 13 1 6 65 75 7 81 42 93

... Fin du Programme! ...
```

Voici les opérations possibles sur une file :

- déterminer la taille de la file : `File.size()`;
- déterminer si la file est vide ou non : `File.empty()` ;
- ajouter une nouvelle donnée : `File.push(element)` ;
- consulter la première donnée enfilée : `File.front()` ;
- consulter la dernière donnée enfilée : `File.back()` ;
- supprimer la première donnée : `File.pop()` .

Les listes avec la STL C++

Une ***liste chaînée*** est une structure de données dans laquelle les objets sont arrangés linéairement. Toutefois, contrairement au tableau, pour lequel l'ordre linéaire est déterminé par les indices, l'ordre d'une liste chaînée est déterminé par un pointeur dans chaque objet. Les listes chaînées fournissent une représentation simple et souple pour les ensembles dynamiques.



Une liste peut prendre différentes formes. Elle peut être chaînée, ou doublement chaînée, triée ou non, circulaire ou non.

```
1
2  /*
3   * File:   main.cpp
4   * Author: ACER
5   *
6   * Created on 23 fÃ©vrier 2017, 19:57
7   */
8
9  #include <iostream>
10 #include <list>
11
12 using namespace std;
13
14 int main()
15 {
16     list<int> ma_liste;
17     ma_liste.push_back(9);
18     ma_liste.push_back(16);
```

```
19      ma_liste.push_back(1);
20      ma_liste.push_front(25);
21      ma_liste.push_front(16);
22      ma_liste.insert(++ma_liste.begin(), 2);
23      // ma_liste contient 16 25 2 9 16 1
24
25      cout << endl;
26      cout << "  1. parcour de la liste dans le premier sens" << endl;
27      cout << "     ----------------------------------------" << endl;
28      cout << "\n   (debut de la liste) -> " ;
29      list <int>:: iterator iter;
30      for(iter=ma_liste.begin(); iter != ma_liste.end(); ++iter)
31          cout << "(" << *iter << ") -> ";
32      cout << "(fin de la liste)" << endl << endl;
33
34      cout << "  2. parcour de la liste dans le deuxieme sens" << endl;
35      cout << "     ----------------------------------------" << endl;
36      cout << "\n   (fin de la liste) -> ";
37      list <int>:: reverse_iterator riter;
38      for(riter=ma_liste.rbegin(); riter != ma_liste.rend(); ++ riter)
39          cout << "(" << *riter << ") -> ";

40      cout << "(debut de la liste)" << endl << endl;
41
42      cout << "  3. La taille de la liste = " << ma_liste.size() << endl << endl;
43
44      cout << "  4. Tri des elements la liste" << endl;
45      cout << "     -------------------------" << endl;
46      ma_liste.sort();
47      cout << "   (debut de la liste) -> " ;
48      for(iter=ma_liste.begin(); iter != ma_liste.end(); ++iter)
49          cout << "(" << *iter << ") -> ";
50      cout << "(fin de la liste)" << endl << endl;
51
52      cout << "  5. Elimination des doubllons" << endl;
53      cout << "     -------------------------" << endl;
54      ma_liste.unique();
55      cout << "\n   (debut de la liste) -> " ;
56      for(iter=ma_liste.begin(); iter != ma_liste.end(); ++iter)
57          cout << "(" << *iter << ") -> ";
58      cout << "(fin de la liste)" << endl << endl;
59
60      cout << "  6. La taille de la liste = " << ma_liste.size() << endl << endl;
61
62      cin.get();
63      return 0;
```

Résultat de l'exécution

```
1. parcour de la liste dans le premier sens
   ----------------------------------------
   (debut de la liste) -> (16) -> (2) -> (25) -> (9) -> (16) -> (1) -> (fin de la liste)

2. parcour de la liste dans le deuxieme sens
   ----------------------------------------
```

```
   (fin de la liste) -> (1) -> (16) -> (9) -> (25) -> (2) -> (16) -> (debut de la liste)

3. La taille de la liste = 6

4. Tri des elements la liste
   -------------------------
   (debut de la liste) -> (1) -> (2) -> (9) -> (16) -> (16) -> (25) -> (fin de la liste)

5. Elimination des doubllons
   -------------------------
   (debut de la liste) -> (1) -> (2) -> (9) -> (16) -> (25) -> (fin de la liste)

6. La taille de la liste = 5
```

Les vecteurs avec la STL C++

La classe *vector* fait partie de la **bibliothèque STL** (Standard Template Library). C'est un tableau dynamique.

```cpp
/*
 * File:   main.cpp
 * Author: ACER
 *
 * Created on 23 février 2017, 22:55
 */
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
bool decroissant (int i,int j) { return (i>j); }
bool croissant (int i,int j) { return (i<j); }
int main() {
   vector<int> vec; // création d'un vecteur d'entiers
   int i;
   cout << endl;
   // Imprression de la taille originale du vecteur
   cout << "  la taille initiale du vecteur = " << vec.size() << endl;
   // mettre 5 valeurs dans le vecteur
   for(i = 0; i < 5; i++)
      vec.push_back(i*i);
   //-------------------------------------------------------------------
   // Imprression de la taille du vecteur
   cout << "  la taille finale du vecteur = " << vec.size() << endl;
   // consulter les 5 valueurs du vecteur 1 façon
   for(i = 0; i < 5; i++){
      cout << "  vec [" << i << "] = " << vec[i] << " " ;
   }
    cout << "    avec la 1 facon" <<endl;
   //-------------------------------------------------------------------
   // utilisation d'un iterator pour accéder aux valeurs
   vector<int>::iterator it ;
   int j=0;
   for(it=vec.begin(); it!=vec.end(); it++)
```

```
35     {
36        cout << "  vec [" << j << "] = " << *it << " " ;
37        j++;
38     }
39     cout << "    avec la 2 facon" << endl;
40     //-----------------------------------------------------------------
41      it = vec.begin(); j=0;
42      while( it != vec.end())
43      {
44        cout << "  vec [" << j << "] = " << *it << " " ;
45        it++;
46        j++;
47      }
48      cout << "    avec la 3 facon" << endl;
49     //-----------------------------------------------------------------
50      sort (vec.begin(), vec.end(), decroissant);
51      j=0;
52      for(it=vec.begin(); it!=vec.end(); it++)
53      {
54        cout << "  vec [" << j << "] = " << *it << " " ;
55        j++;
56      }
57      cout << "    apres le Tri decroissant " << endl;
58      //-----------------------------------------------------------------
59      sort (vec.begin(), vec.end(), croissant);
60      j=0;
61      for(it=vec.begin(); it!=vec.end(); it++)
62      {
63        cout << "  vec [" << j << "] = " << *it << " " ;
64        j++;
65      }
66      cout << "    apres le Tri croissant " << endl;
67     //-----------------------------------------------------------------
68     return 0;
69
```

Résultat de l'exécution

```
la taille initiale du vecteur = 0
la taille finale du vecteur = 5
vec [0] = 0   vec [1] = 1   vec [2] = 4   vec [3] = 9   vec [4] = 16     avec la 1 façon
vec [0] = 0   vec [1] = 1   vec [2] = 4   vec [3] = 9   vec [4] = 16     avec la 2 façon
vec [0] = 0   vec [1] = 1   vec [2] = 4   vec [3] = 9   vec [4] = 16     avec la 3 façon
vec [0] = 16   vec [1] = 9   vec [2] = 4   vec [3] = 1   vec [4] = 0     après le Tri décroissant
vec [0] = 0   vec [1] = 1   vec [2] = 4   vec [3] = 9   vec [4] = 16     après le Tri croissant
```

Exemple avec `find`

```
1  // find example
2  #include <iostream>     // std::cout
3  #include <algorithm>    // std::find
4  #include <list>
5  #include <string>
6
7  using namespace std;
8
9  int main () {
10     std::list<int> myList;
11     myList.push_back(5);
12     myList.push_back(19);
13     myList.push_back(34);
14     myList.push_back(3);
15     myList.push_back(33);
16
17     std::list<int>::iterator iter;
18     iter= find (myList.begin(), myList.end(), 34);
19     if (iter != myList.end())
20         std::cout << "Element found in myList : " << *iter << '\n';
21     else
22         std::cout << "Element not found in myList\n";
23
24     std::list < std::string > myListstr;
25     myListstr.push_back("Systèmes");
26     myListstr.push_back("Experts");
27     myListstr.push_back("Programme");
28     myListstr.push_back("Logique");
29     myListstr.push_back("Prolog");
30
31     std::list<string>::iterator iterstr;
32     iterstr= find (myListstr.begin(), myListstr.end(), "Programme");
33
34     if (iterstr != myListstr.end())
35         std::cout << "Element found in myListstr : " << *iterstr << '\n';
36     else
37         std::cout << "Element not found in myListstr\n";
38
39   return 0;
```